Project 04 实验报告

课程：数字图像处理　　组别：20

实验总览

编号	实验题目	实验要求	主要结果	页码	负责人
04-03	基于从零实现的 2D DFT 进行低通滤波	(a) 实现尺寸与中心均可指定的高斯低通滤波器；(b) 对 Fig. 4.11(a) 低通滤波，得到 Fig. 4.18(c)	DFT 往返误差 1.2×10^{-10}；取 $\sigma = 30$ 复现 Fig. 4.18(c)；缓存频谱使 σ 扫描加速 1.64 倍	2	Y.Surayya
04-04	利用低通图像实现高通滤波	(a) 原图减去 04-03(b) 的低通结果，解释与 Fig. 4.26 的差异；(b) 调整 σ，使结果与 Fig. 4.26(c) 相似	相减与 $1 - H$ 滤波等价（误差 1.1×10^{-10}）；$\sigma = 80$ 时与 Fig. 4.26(c) 相似；保留能量由 41% 降至 15%	11	Y.Surayya
04-05	频域相关：用 DFT 完成模板匹配	用 DFT 实现二维相关并复现教材例 4.11（图 4.41）：把模板图 4.41(b) 在图像图 4.41(a) 中定位，显示相关函数图 4.41(e) 与灰度剖面线图 4.41(f)	峰值 / 均值达 110 的尖锐单峰；峰值处 NCC = 1.0000；DFT 相关与按定义的相关一致且快 12 倍；另验证习题 4.22 与 4.23	19	代骏泽

Project 04-03：基于从零实现的 2D DFT 进行低通滤波

负责人：Y.Surayya

1 实验要求

- **(a)** 实现式 (4.3-7) 中的高斯低通滤波器。必须能够指定所得二维函数的尺寸 $M \times N$ 以及高斯函数中心的二维位置。
- **(b)** 下载 Fig. 4.11(a)（与 Fig. 4.18(a) 相同）并对其进行低通滤波，得到 Fig. 4.18(c)。

所使用的傅里叶变换来自我们在 Project 04-01 中的自实现，处理链路中不使用 `np.fft`。

除必做项之外，我们还研究了：(i) DFT 是否正确以及其速度如何；(ii) 自实现的 radix-2 FFT 是否比矩阵 DFT 更快；(iii) 高斯滤波器与理想滤波器、巴特沃斯滤波器的对比；(iv) 缓存频谱能为 σ 扫描带来多大加速；(v) 上述一切能否在其他图像上正常工作。

2 实验过程

2.1 理论基础

2.1.1 可分离的 2D DFT

对于一幅尺寸为 $M \times N$ 的图像 $f(x, y)$，其二维 DFT 为

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)\, e^{-j2\pi (u\frac{x}{M} + v\frac{y}{N})}.$$

由于核是可分离的，该变换等价于先对每一列做一维 DFT，再对每一行做一维 DFT。借助 DFT 矩阵 $W_N(k, n) = e^{-j2\pi kn/N}$，可写成

$$F = W_M \cdot f \cdot W_N^{T},$$

这样只需两次矩阵乘法，无需任何像素级循环。

2.1.2 逆 DFT

逆变换与前向变换的区别仅在于指数的符号以及一个因子 $1/(MN)$。对输入和输出同时取共轭即可翻转指数符号，因此

$$f = \frac{\overline{\mathrm{DFT}\big(\overline{F}\big)}}{MN}.$$

无需单独实现逆变换函数。

2.1.3 中心化

将 $f(x, y)$ 乘以 $(-1)^{x+y}$ 可将直流（DC）分量移至 $(M/2, N/2)$。对于偶数尺寸的图像，这一步等价于 `fftshift`。滤波器也必须以同一点为中心。

2.1.4 高斯低通滤波器

$$H(u, v) = \exp\left(-\frac{D^2(u, v)}{2\sigma^2} \right), \quad D^2(u, v) = (u - u_0)^2 + (v - v_0)^2.$$

参数 σ 是频域中的宽度。σ 越大，通过的高频越多（模糊越轻）；σ 越小，通过的高频越少（模糊越强）。中心 (u_0, v_0) 是一个参数，默认取 `(M//2, N//2)`。

2.1.5 滤波流程

$$f \to \times (-1)^{x+y} \to \mathrm{DFT} \to \times H(u, v) \to \mathrm{IDFT} \to \times (-1)^{x+y} \to \mathrm{Re} \to g(x, y)$$

即：乘以 $(-1)^{x+y}$，做 DFT，与 $H(u,v)$ 逐元素相乘，做逆 DFT，再乘以 $(-1)^{x+y}$ 并取实部。

2.2 代码组织

文件	作用
`src/dft.py`	`dft_matrix`、`dft2d`、`idft2d`、`center`、`spectrum`、`filter_freq`（Project 04-01 包）
`src/gaussian_filter.py`	`gaussian_lowpass(M, N, sigma, center_uv)`：部分 (a)
`src/utils.py`	`load_image`、`save_fig`、`save_image`：将指向 `data/` 和 `results/` 的路径集中在同一处
`src/main.py`	部分 (b)：对图像滤波并保存结果
`src/extra.py`	扩展实验（第 3 节）
`test/testing_dft.py`	检验 DFT 是否正确（第 2.3 节）
`test/test1_pic.bmp`、`test/test2_pic.bmp`	额外测试图像（第 2.6 节）

图像 `Fig0418.jpeg` 存放在 `data/` 中，所有输出写入 `results/`。项目文件夹自成一体、完全独立。

2.3 DFT 的验证（`test/testing_dft.py`）

在滤波之前，先在真实测试图像上对 DFT 进行了验证。所有图像均以 `float64` 加载，因为 `uint8` 运算会溢出。

- 图像尺寸： 688×688，灰度范围为 0 到 255。
- 往返测试： 将 `idft2d(dft2d(f))` 与 f 比较，最大绝对误差为 1.2×10^{-10}，属于浮点舍入误差水平。
- 中心化频谱： 对数标度的频谱 $S = \log(1 + |F|)$ 的最亮点位于数组中心，能量向边缘递减（图 1）。
- 平均值： 将位于中心的直流分量除以 MN 后与图像均值比较。

```
forward DFT (8, 8): max error = 1.60e-14  ->  PASS
forward DFT (6, 10): max error = 2.80e-14  ->  PASS
forward DFT (31, 20): max error = 1.45e-12  ->  PASS
speed: 256x256 DFT took 0.0140 s
image: (688, 688) min 0.0 max 255.0
round trip on the image: max error = 1.20e-10  ->  PASS
DC / (M*N): 207.25278233166577
image mean: 207.25278233166577
average value: max error = 0.00e+00  ->  PASS
```

两者一致，从而验证了变换与中心化的正确性。

其中 PASS 的定义为：

```python
def check(name, err, tol=1e-8):
    print(f"{name}: max error = {err:.2e}  ->  {'PASS' if err < tol else 'FAIL'}")
```

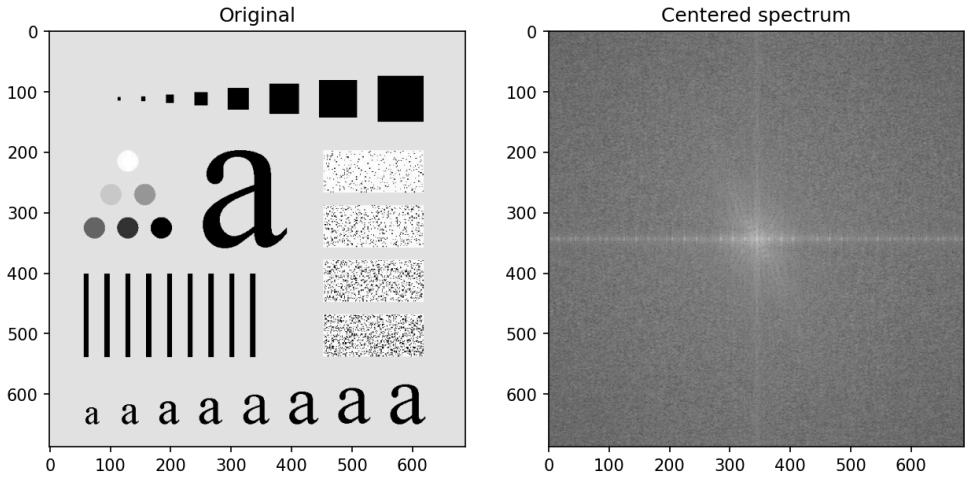


图 1　fig_spectrum.png：原始图像及其中心化的傅里叶频谱。

2.4 部分 (a)：高斯低通滤波器

`gaussian_lowpass(M, N, sigma, center_uv=None)` 利用 `np.indices` 一次性构建完整的 $M \times N$ 数组并返回 $\exp(-D^2/(2\sigma^2))$。尺寸由 `M` 和 `N` 指定，中心由 `center_uv` 指定（默认取数组中央）。

以 $M = N = 688$、$\sigma = 30$ 进行验证：

检查项	预期	实测
`H.shape`	$(688, 688)$	$(688, 688)$
中心处 H	1.0	1.0
角落 $[0, 0]$ 处 H	近似 0	7.889×10^{-58}

另外生成了 `center_uv = (100, 200)` 的滤波器，以表明中心可以指定。其最大值出现在 $(100, 200)$ 处。

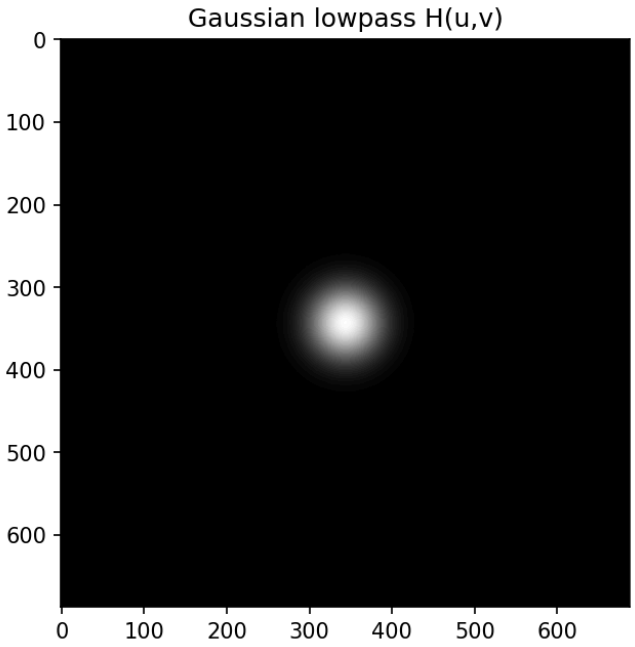


图 2　fig01_0403_H_plot.png：高斯低通滤波器，$\sigma = 30$，已中心化。

2.5 部分 (b)：对 Fig. 4.18(a) 的低通滤波

流程。用 `filter_freq(f, H)` 对图像进行滤波，σ 分别取 5、15、30、80 和 230。将结果与教材中的 Fig. 4.18(c) 对比，选取最接近的一个。

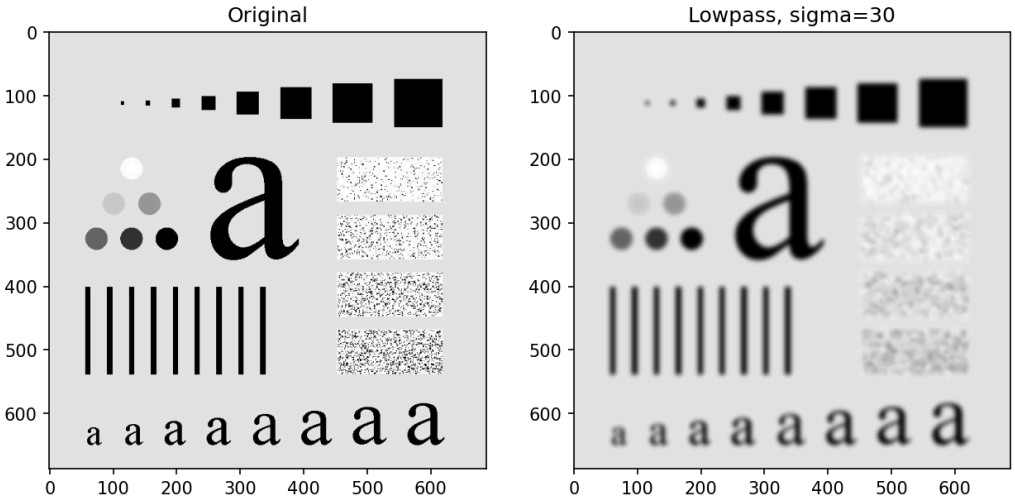


图 3　fig02_0403_lowpass_comparison.png：原始图像（左）与高斯低通结果（右），$\sigma = 30$。



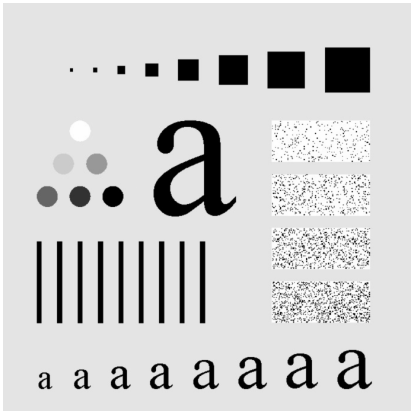
$\sigma = 5$
fig03_0403_lowpass_sigma5.png

$\sigma = 15$
fig04_0403_lowpass_sigma15.png

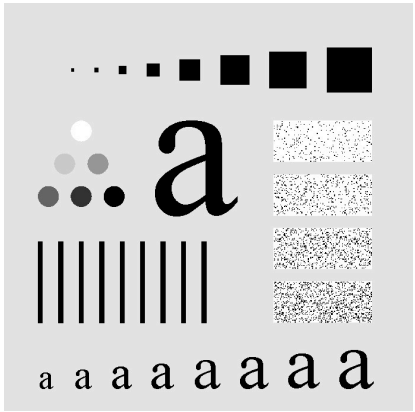
$\sigma = 30$
fig05_0403_lowpass_sigma30.png



$\sigma = 80$
fig06_0403_lowpass_sigma80.png



$\sigma = 230$
fig07_0403_lowpass_sigma230.png



原始图像
Fig0418.jpeg

图 4　fig03–07_0403_lowpass_sigma_*.png：高斯低通结果（$\sigma = 5$、15、30、80、230）与原始图像。

讨论。

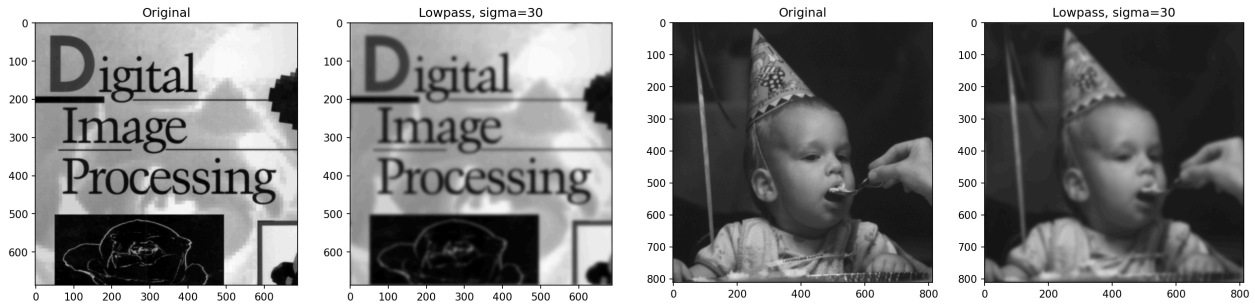
- **σ 的作用。** σ 是 H 在频域中的宽度，因此 σ 小时只保留低频、模糊较强；σ 大时结果接近原图。

- 图像为何变模糊。边缘、细线条和小字符由高频分量承载。去除高频后，锐利的跳变变为平缓的过渡。
- 无振铃。 高斯滤波器没有锐利的截止，且其逆变换仍为正的高斯函数，因此不会产生振铃（对比第 3.2 节）。
- 中心化。 H 的中心位于 $(M/2, N/2)$，与乘以 $(-1)^{x+y}$ 后直流分量的位置相同。若两者不匹配，就会滤除错误的频率。
- 实部输出。 舍入误差留下的虚部可以忽略，因此最后取实部。

2.6 推广到其他图像

在不改动算法的情况下，将同一套代码用于另外两幅测试图像（`test/test1_pic.bmp` 与 `test/test2_pic.bmp`）。输出结果保存在 `test/test1_results/` 与 `test/test2_results/` 中。

图像	尺寸	内容	结果
`test1_pic.bmp`	688×688	《数字图像处理》封面灰度图	σ 越小模糊越强，无伪影
`test2_pic.bmp`	688×688	灰度教材测试图：一个头戴图案纸生日帽的幼童	行为相同



`test/test1_results/fig02_0403_lowpass_comparison.png` `test/test2_results/fig02_0403_lowpass_comparison.png`

图 5　额外测试图像的低通滤波结果，左：`test1_pic.bmp`，右：`test2_pic.bmp`。（其余结果见 `test/test1_results/` 与 `test/test2_results/` 文件夹）

由于 DFT 矩阵由图像自身的高度和宽度构建，非正方形尺寸以及非 2 的幂次尺寸都无需填充或缩放即可处理。

3 扩展实验与性能

由于必做部分较为简单，我们还进一步研究了该方法的行为特性。所有实验位于 `src/extra.py`。每个计时均取 3 次运行中的最佳值以降低噪声。

3.1 自实现的 radix-2 FFT 与矩阵 DFT 的对比

思路。矩阵 DFT 的代价约为 $O(n^3)$（对于 $n \times n$ 的图像），而 radix-2 FFT 约为 $O(n^2 \log n)$。我们沿最后一个轴实现了递归 radix-2 FFT（按偶、奇样本拆分，用旋转因子合并），并先后应用于行与列。它要求尺寸为 2 的幂。

精度。 FFT 与矩阵 DFT 的偏差最大为 2.0×10^{-11}（在随机 64×128 数组上的最大绝对差）。

速度。 在 $n \times n$ 随机数组上测得的时间：

n	矩阵 **DFT**（s）	自实现 **FFT**（s）	矩阵 / **FFT**
64	0.0006	0.0017	0.34
128	0.0020	0.0051	0.39
256	0.0124	0.0162	0.76
512	0.0475	0.0609	0.78
1024	0.1988	0.2868	0.69
2048	1.2045	1.4394	0.84

双对数坐标下的斜率：矩阵 DFT 为 2.14，FFT 为 1.81。

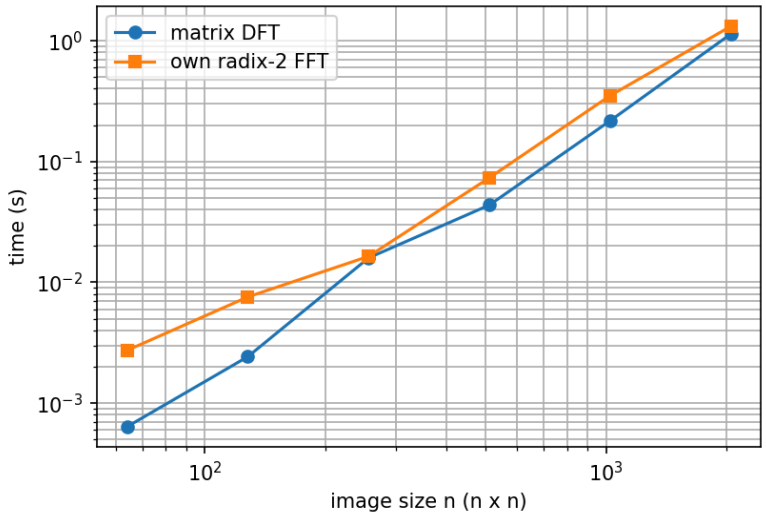


图 6　ext1_speed.png：矩阵 DFT 与自实现 radix-2 FFT 的对比。

发现。

- 在所有测试尺寸上矩阵 DFT 都更快（比值均小于 1），因此此处 FFT 并不划算。直到 2048 仍未观察到交叉点。
- FFT 增长更慢（斜率 1.81 对 2.14），因此总体上随着 n 增大，比值从 0.34 上升至 0.84。由此可以预期 FFT 在更大尺寸下会胜出，但我们没有实际测量。
- 矩阵 DFT 的斜率远低于理论值 3。矩阵方法利用了 NumPy 优化过的多线程矩阵乘法，而我们的 FFT 是用 Python 编写的，包含大量小数组操作。FFT 的增长速率更低，但常数开销更高。
- 个别比值存在波动（例如 $n = 1024$ 时为 0.69，低于 $n = 512$ 时的 0.78），我们将其归因于计时噪声，因此只依据总体趋势得出结论。

这也解释了为什么主项目仍采用矩阵 DFT：对于此处使用的图像尺寸，它更简单且并不更慢。

3.2 理想、巴特沃斯与高斯低通滤波器

三种滤波器均采用相同的截止频率 $D_0 = 30$：理想滤波器（半径 D_0 以内为 1，以外为 0）、2 阶巴特沃斯滤波器，以及 $\sigma = D_0$ 的高斯滤波器。

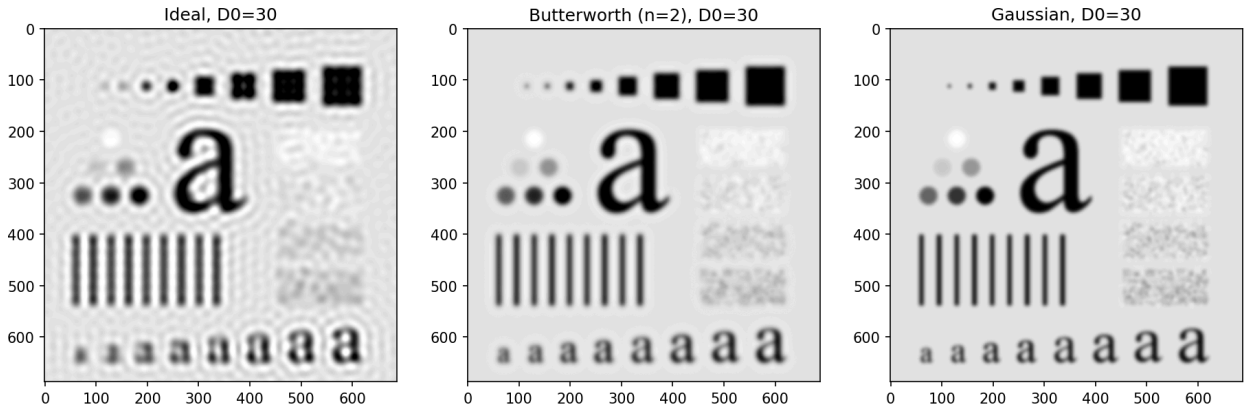


图 7　ext2_filter_types.png：理想、巴特沃斯（$n = 2$）与高斯低通滤波器，截止频率 $D_0 = 30$。

理想滤波器在边缘和大字母周围出现波纹（振铃），巴特沃斯滤波器有轻微振铃，高斯滤波器则完全没有。理想滤波器出现振铃是预期之中的，因为其锐利的截止在空间域对应一个具有振荡旁瓣的 sinc 形核。高斯滤波器没有这样的旁瓣，这正是本项目选用它的原因。

3.3 σ 扫描中的频谱缓存

思路。 图像的正向 DFT 与 σ 无关。因此对多个 σ 值进行扫描时，只需一次正向 DFT，之后每个 σ 仅需一次乘法与一次逆 DFT。

结果。 对 8 个 σ 值，朴素扫描耗时 1.637 s，缓存扫描耗时 0.998 s，加速比为 **1.64**。两种方法得到的图像完全一致（最大差异为 0.0）。

朴素扫描需要 16 次变换（8 次正向、8 次逆变换），缓存扫描只需 9 次（1 次正向、8 次逆变换）。因此加速比的上限为 $16/9 \approx 1.78$。实测值略低，因为构建滤波器、乘法以及去中心化步骤并不会缩短。

同样的思路也让交互式调节 σ 变得廉价：完成第一次变换后，每尝试一个新的 σ 只需一次逆 DFT。

4 关键代码

4.1 `src/dft.py`

```python
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

def dft_matrix(N):
    n = np.arange(N)
    k = n.reshape(N, 1)
    return np.exp(-2j * np.pi * k * n / N)

def dft2d(f):
    M, N = f.shape
    return dft_matrix(M) @ f @ dft_matrix(N).T

def idft2d(F):
    M, N = F.shape
    return np.conj(dft2d(np.conj(F))) / (M * N)

def center(f):
    M, N = f.shape
    x, y = np.indices((M, N))
    return f * (-1.0) ** (x + y)

def spectrum(F):
```

```
    S = np.log(1 + np.abs(F))
    return 255 * S / S.max()

def filter_freq(f, H):
    F = dft2d(center(f))
    G = F * H
    g = idft2d(G)
    return np.real(center(g))
```

4.2 src/gaussian_filter.py

```
import numpy as np

def gaussian_lowpass(M, N, sigma, center_uv=None):
    if center_uv is None:
        center_uv = (M // 2, N // 2)
    u0, v0 = center_uv
    u, v = np.indices((M, N))
    D2 = (u - u0)**2 + (v - v0)**2
    return np.exp(-D2 / (2 * sigma**2))
```

4.3 src/main.py（必做部分）

```
from dft import *
from gaussian_filter import *
from utils import *

f = load_image()
M, N = f.shape
H = gaussian_lowpass(M, N, sigma=30)

g = filter_freq(f, H)

for sigma in [5, 15, 30, 80, 230]:
    g = filter_freq(f, gaussian_lowpass(M, N, sigma))
    save_image(g, f"lowpass_sigma{sigma}.png")
```

4.4 src/extra.py（radix-2 FFT 与缓存扫描）

```
def fft_last_axis(x):
    n = x.shape[-1]
    if n == 1:
        return x.astype(complex)
    even = fft_last_axis(x[..., 0::2])
    odd = fft_last_axis(x[..., 1::2])
    w = np.exp(-2j * np.pi * np.arange(n // 2) / n)
    return np.concatenate([even + w * odd, even - w * odd], axis=-1)

def fft2d(a):
    return fft_last_axis(fft_last_axis(a).T).T

def sweep_cached():
    Fc = dft2d(center(f))                  # forward DFT only once
    for s in sigmas:
        np.real(center(idft2d(Fc * gaussian_lowpass(M, N, s))))
```

5 实验结果

我们从零实现了可分离矩阵形式的二维 DFT，并以 1.2×10^{-10} 的往返误差、正确的中心化频谱和一致的平均值对其进行了验证。实现了尺寸与中心均可指定的高斯低通滤波器，取 $\sigma = 30$ 时可复现 Fig. 4.18(c) 的模糊效果。同一套代码无需修改即可用于其他图像。在扩展实验中，自实现的 radix-2 FFT 精度达到 2×10^{-11}，但在 2048×2048 以内比优化过的矩阵 DFT 慢（其增长率表明更大尺寸下它将胜出）；理想滤波器出现高斯滤波器所没有的振铃；缓存频谱使 σ 扫描获得了 1.64 倍的加速且结果完全一致。

Project 04-04：利用低通图像实现高通滤波

负责人：Y.Surayya

1 实验要求

- **(a)** 将项目 04-03(b) 的结果图像从原图中减去，得到锐化图像，如式 (4.4-14) 所示。所得图像与图 4.26 中的高斯高通结果并不相似，请解释原因。
- **(b)** 调整高斯低通滤波器的方差，使图像相减得到的结果与图 4.26(c) 相似，并解释该结果。

本项目使用的二维傅里叶变换是项目 04-01 中自行实现的版本，处理流程中不使用 `np.fft`。

除必做内容外，我们还研究了：(i) 图像相减是否与频域高通滤波严格等价；(ii) 如何加快 σ 扫描；(iii) 对每个 σ，高通结果中还保留了多少图像能量；(iv) 高提升滤波；(v) 该方法对其他图像以及非方形、含奇数边长的图像是否同样适用。

2 实验过程

2.1 理论基础

2.1.1 通过相减得到高通（式 4.4-14）

非锐化掩模(unsharp mask)是用图像减去它的模糊副本得到的。在频域中，这相当于从原图中减去低通滤波后的图像：

$$f_{\text{hp}}(x,y) = f(x,y) - f_{\text{lp}}(x,y).$$

由于 DFT 是线性的，这等价于用下面的滤波器对 f 滤波：

$$H_{\text{hp}}(u,v) = 1 - H_{\text{lp}}(u,v).$$

取高斯低通 $H_{\text{lp}}(u,v) = \exp\!\left(-\frac{D^2}{2\sigma^2}\right)$ 时，对应的高通滤波器在中心处为 0（去掉直流分量），在远离中心处趋近于 1。因为 $H_{\text{lp}}(0,0) = 1$，低通图像保留了 f 的均值，所以差图像 $f - f_{\text{lp}}$ 的均值为零：平坦区域接近 0，边缘处既有正值也有负值。

2.1.2 σ 的作用

σ 是高斯函数在频域中的宽度，因此它与教材中高斯滤波器的截止频率 D_0 作用相同。图 4.26 的 (a)、(b)、(c) 三个子图分别对应 $D_0 = 15, 30, 80$ 的高斯高通结果。所以图 4.26(c) 对应较大的 σ，约为 80。

2.1.3 高提升滤波（扩展）

在减去低通图像之前，先把原图乘以常数 $A \geq 1$，得到

$$f_{\text{hb}}(x,y) = A f(x,y) - f_{\text{lp}}(x,y) = (A-1) f(x,y) + f_{\text{hp}}(x,y).$$

$A = 1$ 时就是普通的高通图像。A 越大，加回的原图成分越多，结果看起来就像边缘被增强后的原图。

2.2 代码组织

文件	作用
src/dft.py	自行实现的二维 DFT:dft2d、idft2d、center、spectrum、filter_freq（复制自 04-03）
src/gaussian_filter.py	gaussian_lowpass(M, N, sigma, center_uv)（复制自 04-03）
src/utils.py	load_image、save_fig、save_image（复制自 04-03）
src/main.py	(a)、(b) 两个必做部分
src/extra.py	扩展实验（第 3 节）
test/	额外测试图像及其结果（第 2.5 节）

图像 Fig0418.jpeg（尺寸 688 × 688）存放在 data/ 中，所有输出都保存到 results/。本文件夹不依赖任何其他项目文件夹。

所有运算都使用 float64。带符号的差图像 $f - g$ 只在显示和保存时才被截断或缩放，绝不在相减之前处理。

2.3 部分 (a)：减去 04-03(b) 的结果

方法。 用 04-03(b) 中选定的 σ（即 $\sigma = 30$）计算低通图像 g，再得到高通图像 $f_{\text{hp}} = f - g$。

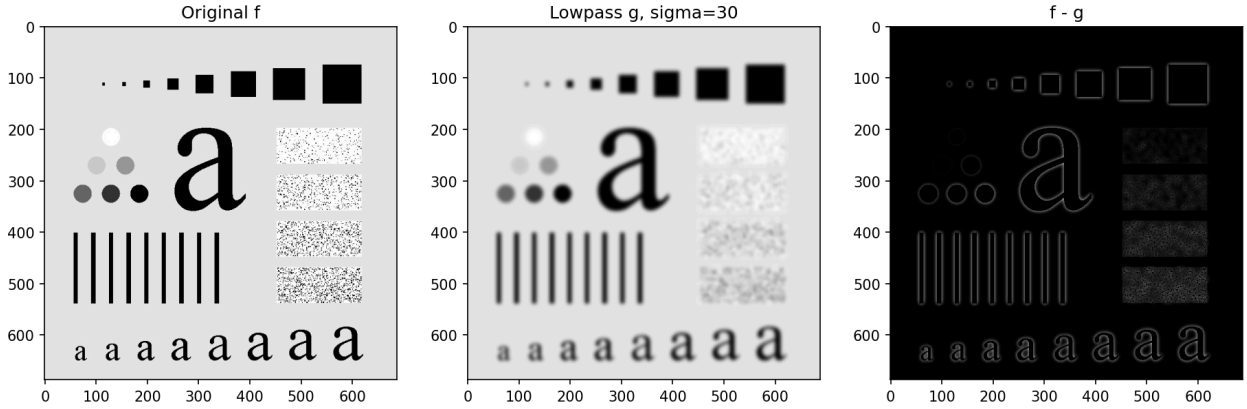


图 8　fig01_0404_a.png：原图、低通结果($\sigma = 30$)及其差 $f - g$。

为什么结果与图 **4.26** 不相似。

1. 运算本身是相同的，差异只来自参数和显示。第 3.1 节的数值结果表明，$f - g$ 与用 $H_{\text{hp}} = 1 - H_{\text{lp}}$ 滤波的结果一致，最大误差约为 1.1×10^{-10}。取相同 D_0 的高斯高通会得到完全相同的图像。因此结果与图 4.26 不同，不是因为“相减”与“高通滤波”是两种不同的运算，而是因为参数与显示方式不同。
2. **σ** 当初是为了另一个目的选的。 在 04-03(b) 中，选 $\sigma = 30$ 是为了让低通图像接近图 4.18(c) 的模糊效果，这对应频域中较窄的低通。把它减掉之后，剩下的是很宽的一段频带，所以 $f - g$ 仍保留了大量图像结构（第 3.3 节：约 27% 的图像能量），边缘又粗又亮。图 4.26(c) 对应的 $D_0 = 80$ 要大得多，只留下细边缘（约 15% 的能量）。
3. 显示方式的缩放。差图像均值为零且含有负值。线性拉伸(min-max)会把平坦区域显示成中灰色，而把负值截断则显示成黑色。教材中的图是深色背景，因此即使数据完全相同，显示方式也会让外观差别很大。
4. 动态范围。 保留下来的细节相对于完整的 0 到 255 范围很小，不重新缩放的话图像显得又暗又淡。

2.4 部分 (b)：调整 σ

步骤。 让低通 σ 取 15、30、80，分别对应图 4.26(a)、(b)、(c) 的 D_0。每个结果用两种方式显示：min-max 线性拉伸，以及把负值截断为黑色。

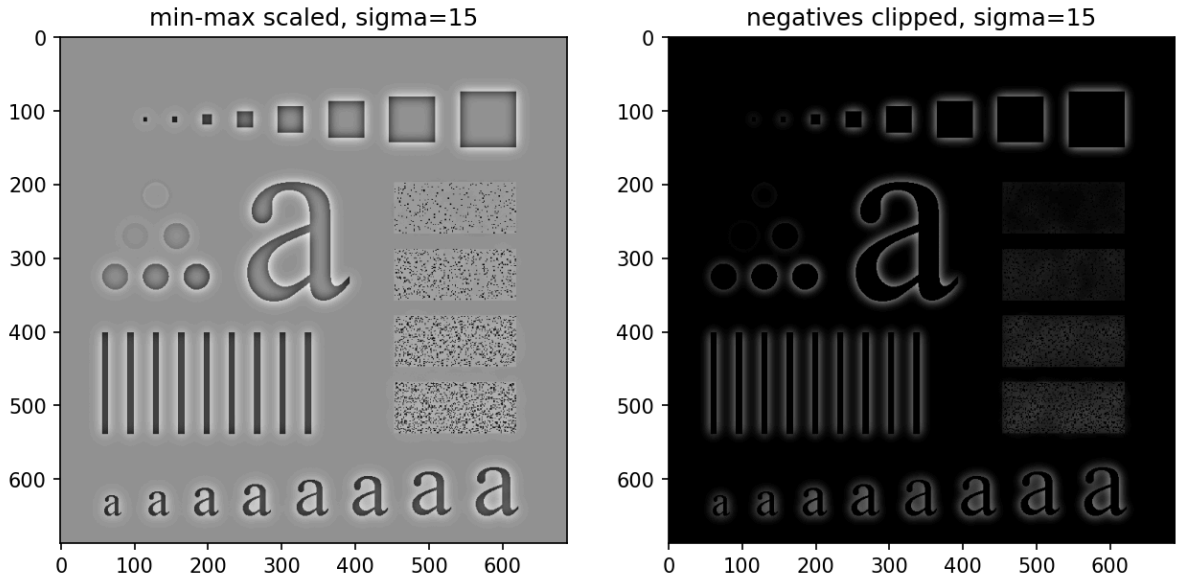


图 9　fig02_0404_sigma15.png：$\sigma = 15$ 时的 $f -$ 低通结果（左：min-max 拉伸；右：负值截断）。

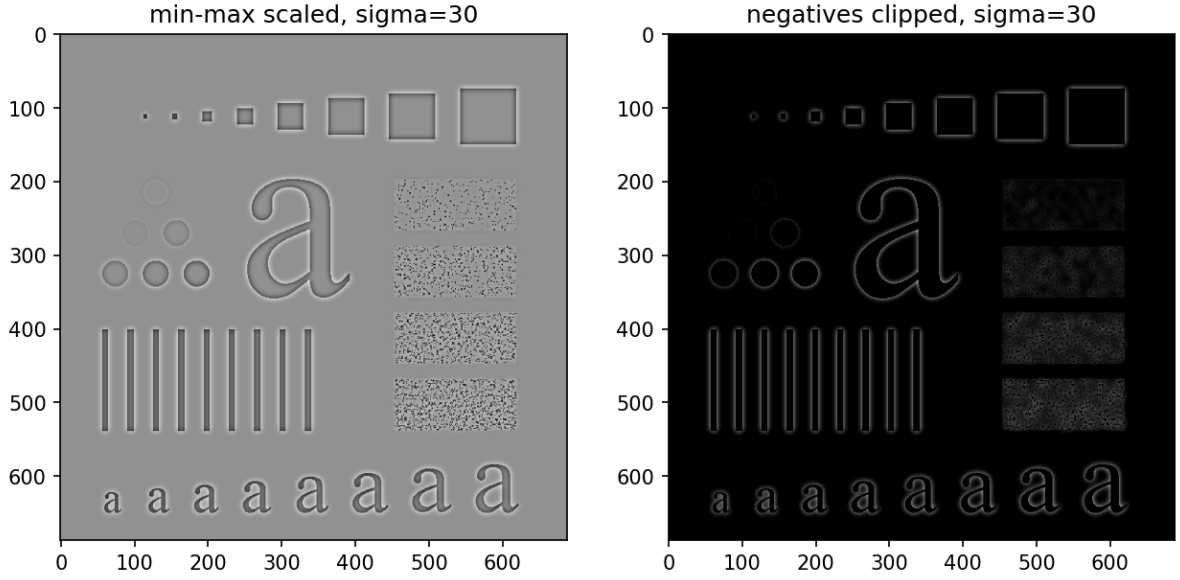


图 10　fig03_0404_sigma30.png：$\sigma = 30$ 时的 $f -$ 低通结果（左：min-max 拉伸；右：负值截断）。

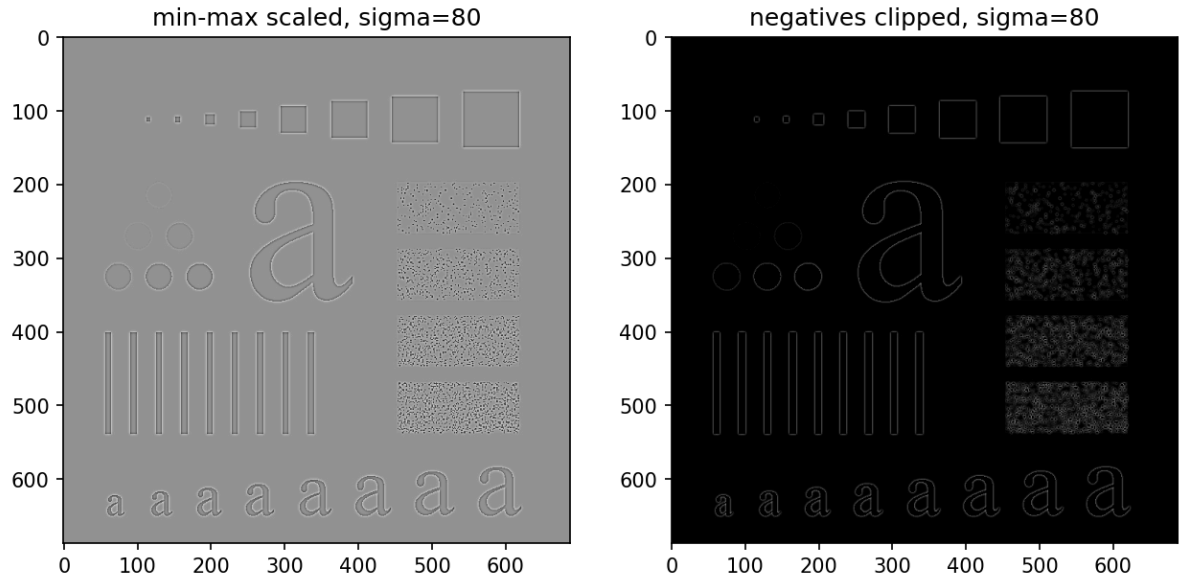


图 11　fig04_0404_sigma80.png：$\sigma = 80$ 时的 $f -$ 低通结果（左：min-max 拉伸；右：负值截断）。

取 $\sigma = 80$ 时的结果与图 4.26(c) 相似：负值截断的显示中背景为暗色，只在大字母、方块等物体的边缘留下细而淡的轮廓，小的结构只剩微弱痕迹。此时高通结果只保留了约 15% 的图像能量（第 3.3 节），而 $\sigma = 15$ 时为 41%。

解释。

- 频域角度。σ 越大，高斯低通越宽，g 越接近原图，$f - g$ 就只保留最高的频率。输出又暗又细、边缘微弱，与图 4.26(c) 一致。
- 空域角度。频域中很宽的高斯对应空域中很窄的高斯，对图像只有轻微的模糊。减去一个只被轻微模糊的副本，只会留下很窄的边缘过渡。
- $\boldsymbol{\sigma}$ 较小时 模糊严重，相减后保留的是粗而亮的结构和大部分图像内容，如图 4.26(a)。
- 显示方式。截断负值的显示与教材的深色背景一致；min-max 拉伸则因为 0 被映射成中灰而显得发灰。

2.5 对其他图像的推广

同样的流程在 `test/` 中的额外测试图像上运行，算法没有任何改动。结果保存在 `test/test_1_results/` 和 `test/test_2_results/` 中，输出文件与 `results/` 相同。

图像	尺寸	内容	结果
`test_1.bmp`	688×688	《数字图像处理》封面灰度图	σ 大时只保留细边缘，σ 小时保留粗结构
`test_2_results.bmp`	688×688	灰度图：一个头戴图案纸生日帽的幼童	表现相同

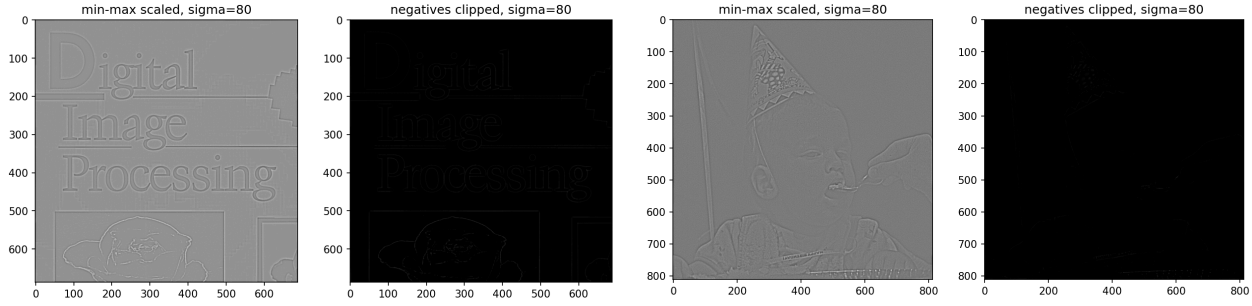


图 12　test/test_1_results/ 与 test/test_2_results/ 中的 fig04_0404_sigma80.png：额外测试图像的高通结果（$\sigma = 80$；左：测试图像 1，右：测试图像 2）。

σ 的变化趋势对每幅图像都相同：σ 越大，只保留越细的边缘。至于哪个 σ 看起来像图 4.26(c)，则取决于图像本身：既取决于图像含有多少细节，也取决于图像尺寸（σ 以频率采样点为单位，同一个 σ 在不同尺寸的图像中覆盖的频谱比例不同）。我们的两幅测试图像尺寸均为 688×688，是方形，但边长不是 2 的幂（第 3.5 节另外检验了非方形图像）。

3 扩展实验与性能

必做内容比较简单，因此我们还研究了该方法的其他性质。所有实验都在 `src/extra.py` 中，每项计时取 3 次运行中的最好值。

3.1 相减与直接高通滤波

取 $\sigma = 30$，用两种方式计算高通：`f - filter_freq(f, H)` 与 `filter_freq(f, 1 - H)`。两者的最大绝对差为 $\mathbf{1.10 \times 10^{-10}}$，处于浮点舍入误差的水平。这证明了减去低通图像与施加 $1 - H$ 是同一种运算，也是第 2.3 节解释的依据。直接形式只需一次正变换和一次逆变换，不需要再做减法。

3.2 更快的 σ 扫描

正变换与 σ 无关。对 8 个 σ 值(5、10、15、20、30、50、80、100)的扫描，朴素方法对每个值都做一次正变换和一次逆变换（共 16 次变换）；缓存方法只算一次中心化频谱，然后对每个 σ 乘以 $1 - H$ 并做一次逆变换（共 9 次变换）。加速比的上界为 $16/9 \approx 1.78$。

方法	时间（**s**）
朴素（每个 σ 做正变换和逆变换）	1.786
缓存频谱	1.121
加速比	1.59

两种方法最后都做同样的反中心化和取实部，所以比较是公平的。实测加速比 1.59 略低于 1.78，因为构造滤波器、乘法以及反中心化这些步骤并没有被节省。

3.3 高通结果中还保留了多少图像？

为了用数字而不是凭眼睛来选 σ，我们计算了高通之后保留的图像能量比例：

$$E(\sigma) = \frac{\sum f_{\mathrm{hp}}^2(x,y)}{\sum \left(f(x,y) - \overline{f}\right)^2},$$

其中减去均值 $\overline{f}$ 是因为高通去掉了直流分量。

σ	**5**	**10**	**15**	**20**	**30**	**50**	**80**	**100**
保留的能量	0.6929	0.5185	0.4147	0.3485	0.2693	0.1988	0.1471	0.1219

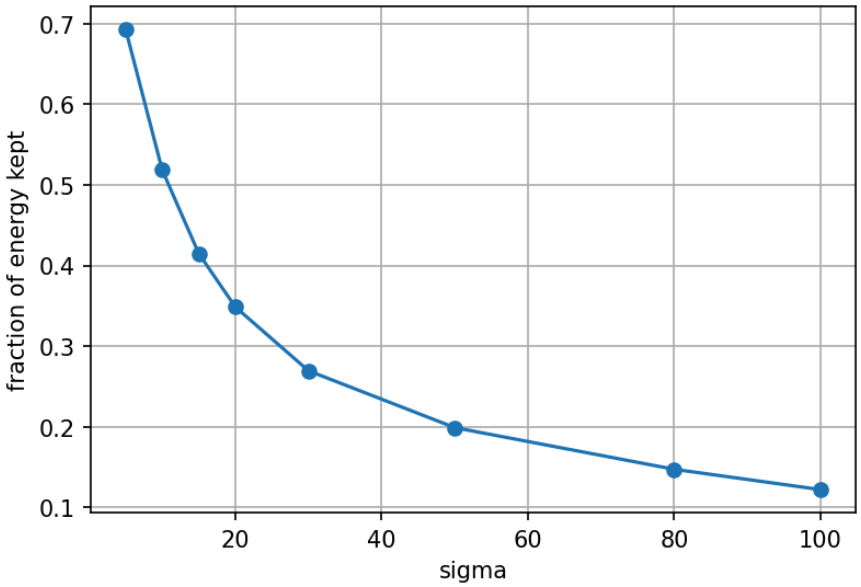


图 13　ext_energy.png：高通所保留的图像能量比例随 σ 的变化。

曲线随 σ 增大而单调下降，这与“输出越来越暗、越来越淡”的视觉印象一致，并给出了一个描述输出“有多高通”的数字。$\sigma = 80$ 时保留的能量为 0.147，约为 $\sigma = 15$ 时(0.415)的 0.35 倍。曲线在 σ 较大时下降变缓：从 $\sigma = 50$ 到 100，能量只从 0.199 降到 0.122。

3.4 高提升滤波

用 $f_{\text{hb}} = Af - f_{\text{lp}}$，取 $A = 1, 1.5, 2$，得到保留原图内容并增强边缘的锐化图像。$A = 1$ 就是 (a) 部分的普通高通图像。A 越大，结果越接近边缘被放大的原图。显示时大于 255 的值被截断。

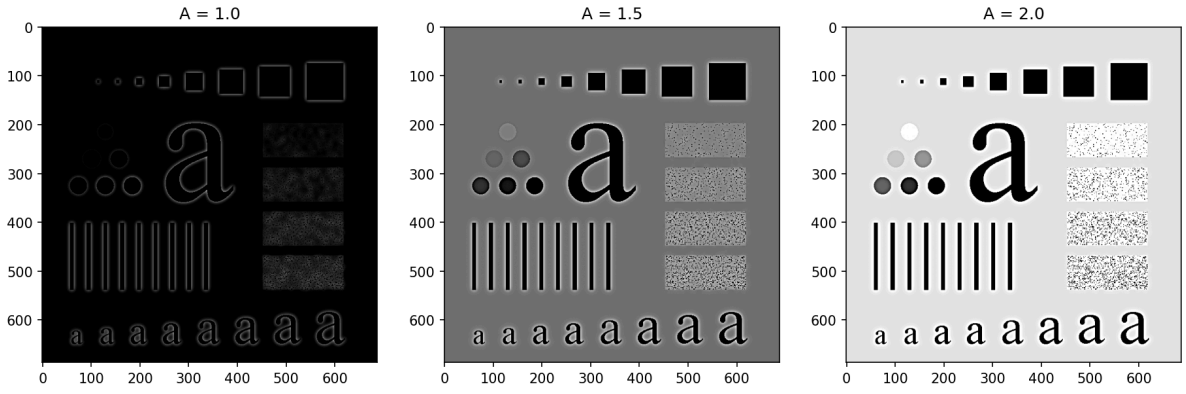


图 14　ext_highboost.png：高提升滤波($\sigma = 30$；$A = 1, 1.5, 2$)。

$A = 1$ 时结果只含边缘，背景为黑色；随着 A 增大，原图的灰度内容被加回，$A = 2$ 时已经是边缘被增强的原图。因为 $f_{\text{hb}} = (A-1)f + f_{\text{hp}}$，$A$ 越大亮区越容易超过 255 而被截断。

3.5 其他图像尺寸

在一幅非方形的裁剪图（尺寸 671 × 648，其中 671 为奇数）上重复了第 3.1 节的等价性检验，最大差值为 $\mathbf{1.33 \times 10^{-10}}$，说明该方法不依赖方形或 2 的幂次的尺寸。对于奇数边长，$(-1)^{x+y}$ 对应的移位量 $M/2$ 不是整数，直流分量不会恰好落在某个整数频率点上，因此中心化只是近似的，需要精确居中时应相应设置滤波器中心；但两种计算方式使用的是同一套线性运算，等价性本身不受影响。

4 关键代码

4.1 src/main.py

```
from dft import *
from gaussian_filter import *
from utils import *
import numpy as np

f = load_image()
M, N = f.shape

def scale(a):
    return 255 * (a - a.min()) / (a.max() - a.min())

# Part(a): org.im - 04-03(b)
fig_idx = 1
sigma_a = 30   # 04-03(b)
g = filter_freq(f, gaussian_lowpass(M, N, sigma_a))
hp = f - g

plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1); plt.imshow(f, cmap="gray", vmin=0, vmax=255); plt.title("Original f")
plt.subplot(1, 3, 2); plt.imshow(g, cmap="gray", vmin=0, vmax=255); plt.title(f"Lowpass g,
sigma={sigma_a}")
plt.subplot(1, 3, 3); plt.imshow(np.clip(hp, 0, 255), cmap="gray", vmin=0, vmax=255); plt.title("f - g")
save_fig(f"fig{fig_idx:02d}_0404_a.png")
plt.show()
fig_idx += 1

# Part(b): varing sigma
for s in [15, 30, 80]:
    hp = f - filter_freq(f, gaussian_lowpass(M, N, s))

    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1); plt.imshow(scale(hp), cmap="gray"); plt.title(f"min-max scaled, sigma={s}")
    plt.subplot(1, 2, 2); plt.imshow(np.clip(hp, 0, 255), cmap="gray", vmin=0, vmax=255);
plt.title(f"negatives clipped, sigma={s}")
    save_fig(f"fig{fig_idx:02d}_0404_sigma{s}.png")
    plt.show()
    fig_idx += 1
```

4.2 src/extra.py（实验核心部分）

```
# equivalence: f - lowpass  ==  filtering with (1 - H)
H = gaussian_lowpass(M, N, sigma)
hp_subtract = f - filter_freq(f, H)
hp_direct = filter_freq(f, 1 - H)
print(np.max(np.abs(hp_subtract - hp_direct)))

# cached sigma sweep: forward DFT only once
def sweep_cached():
    Fc = dft2d(center(f))
    for s in sigmas:
        np.real(center(idft2d(Fc * (1 - gaussian_lowpass(M, N, s)))))

# energy kept
total = np.sum((f - f.mean()) ** 2)
hp = np.real(center(idft2d(Fc * (1 - gaussian_lowpass(M, N, s)))))
kept = np.sum(hp ** 2) / total
```

```
# high-boost
hb = np.clip(A * f - g, 0, 255)
```

5 实验结果

用原图减去高斯低通图像，恰好就是用高斯高通滤波器 $1 - H_{\text{lp}}$ 滤波，所以 (a) 部分的结果与图 4.26 不同，只是由于 04-03(b) 中选的 σ（由那里想要的模糊程度决定）以及显示缩放造成的。把 σ 增大到约 80，就得到了类似图 4.26(c) 的结果：深色背景上又细又淡的边缘。除必做部分外，我们还用数值验证了这种等价性（误差 1.1×10^{-10}），通过缓存频谱降低了 σ 扫描的开销（加速比 1.59，上界 1.78），测量了高通中残留的能量随 σ 的变化（$\sigma = 15$ 时 41%，$\sigma = 80$ 时 15%），实现了高提升滤波，并确认该方法对其他图像以及非方形、含奇数边长的图像同样适用。

Project 04-05：频域相关

负责人：代骏泽

1 实验要求

- 用 DFT 实现二维相关：

$$g(x,y) = \mathrm{IDFT}\Big(F(u,v)\cdot\overline{T(u,v)}\Big)$$

  ，即把图像与模板都做傅里叶变换，取模板变换的复共轭后相乘，再求傅里叶反变换。
- 复现教材例 4.11（图 4.41）：把模板图 4.41(b) 在图像图 4.41(a) 中定位，显示相关函数图 4.41(e)，并画出通过其最高值的水平灰度剖面图 4.41(f)。

所使用的二维傅里叶变换来自我们在 Project 04-01 中的自实现，处理链路中不使用 `np.fft`。

除必做项之外，我们还研究了：(i) 直接（按定义）相关与 DFT 相关是否一致，以及 DFT 快多少；(ii) 相关的时间如何随图像尺寸增长；(iii) 图像均值不为零时朴素相关为什么会失效；(iv) 同一套代码能否定位另一个模板；(v) 非方形、含奇数边长的图像；(vi) 习题 4.22 与 4.23。

2 实验过程

2.1 理论基础

2.1.1 相关的定义

两个函数 f 与 t 的相关（correlation）定义为

$$f(x,y)\star t(x,y) = \sum_{s,t} f(s,t)\,t(s-x,t-y),$$

等价地（对实函数）可写成滑动点积：$g(x,y)$ 是模板 t 平移到 (x,y) 时与原图的点积。相关与卷积的区别只在于：卷积会把模板翻转，相关不会。因此相关适合做“模板匹配”：模板与图中某处完全一致时，相关在该处取最大值。

2.1.2 用 DFT 计算相关

沿用卷积定理的思想（式 4.6.31），相关的傅里叶变换满足

$$\mathcal{F}(f\star t) = \overline{F(u,v)}\cdot T(u,v),$$

教材使用的形式是

$$g(x,y) = \mathrm{IDFT}\Big(F(u,v)\cdot\overline{T(u,v)}\Big),$$

即对图像取正变换，对模板取正变换后复共轭，二者逐点相乘再做反变换。两种写法只差一个整体共轭，而相关函数关于中心对称的特性使它们得到的峰值位置互为镜像（第 3.6 节会给出数值验证）。

2.1.3 零填充与最小尺寸

为了让 DFT 计算的循环相关等于线性相关，必须先把图像和模板分别零填充到至少

$$P \ge A + C - 1, \quad Q \ge B + D - 1,$$

其中 $A\times B$ 是图像尺寸、$C\times D$ 是模板尺寸。例 4.11 中 $A=B=256$、$C=38$、$D=42$，故 $P=293$、$Q=297$。教材把尺寸取整为 298×298，我们使用精确的最小值。

2.1.4 与频域滤波的区别

频域滤波时要先乘 $(-1)^{x+y}$ 把直流分量搬到中心，滤波器也以中心为准。相关不是平移不变的滤波操作，不能这样做：我们直接把零填充后的数组送入 DFT，直流分量留在 $(0,0)$，最后也不做去中心化。这样相关函数的峰值出现在模板左上角所落的位置上，与教材从延拓图像上读出的点是同一个点。

2.2 代码组织

文件	作用
`src/dft.py`	自实现的二维 DFT：`dft2d`、`idft2d`、`center`、`spectrum`（Project 04-01）
`src/correlation.py`	`pad_to`、`circular_correlate`、`linear_correlate`、`find_target`、`match_at`
`src/utils.py`	`load_image`、`save_fig`、`save_image`
`src/make_example_image.py`	构造例 4.11 的输入图像与模板
`src/main.py`	必做部分：定位模板、画相关函数与剖面线
`src/extra.py`	扩展实验（第 3 节）
`test/testing_correlation.py`	验证相关是否正确（第 2.4 节）
`test/run_tests.py`	同一流程在其他测试图像上的运行（第 2.5 节）

2.2.1 关于素材

附件提供的 `4.41a.jpg` 是图 4.41(a) 的一个裁剪——只保留了 “UTK” 三个字母，而 `4.41b.png` 就是其中的 “T” 本身（尺寸 44×49，不是例 4.11 所说的 38×42），且并未落在图像正中。为忠实复现例 4.11（256 ×256 图像、模板是图像的复制且位于中部），`make_example_image.py` 把这两个裁剪重新排布为：

- `data/example_a.png`：256×256 黑底，把 “UTK” 块放在中央；
- `data/example_b.png`：从 `example_a.png` 中切出的 “T” 模板。

`main.py` 同时处理两组素材：重排后的例 4.11 素材（`example_a/b`）与附件原素材（`Fig0441a/b`）。

2.3 部分 (a)：定位模板

流程。读入图像 f 与模板 t，零填充到 $P \times Q = 299 \times 304$，按上式求相关 g，取最大值的位置即为模板所在处。

素材	图像	模板	峰值位置	峰值 / 均值
`example_a/b`	256×256	44×49	$(106, 97)$	110.2
`Fig0441a/b`	256×256	44×49	$(106, 102)$	63.7

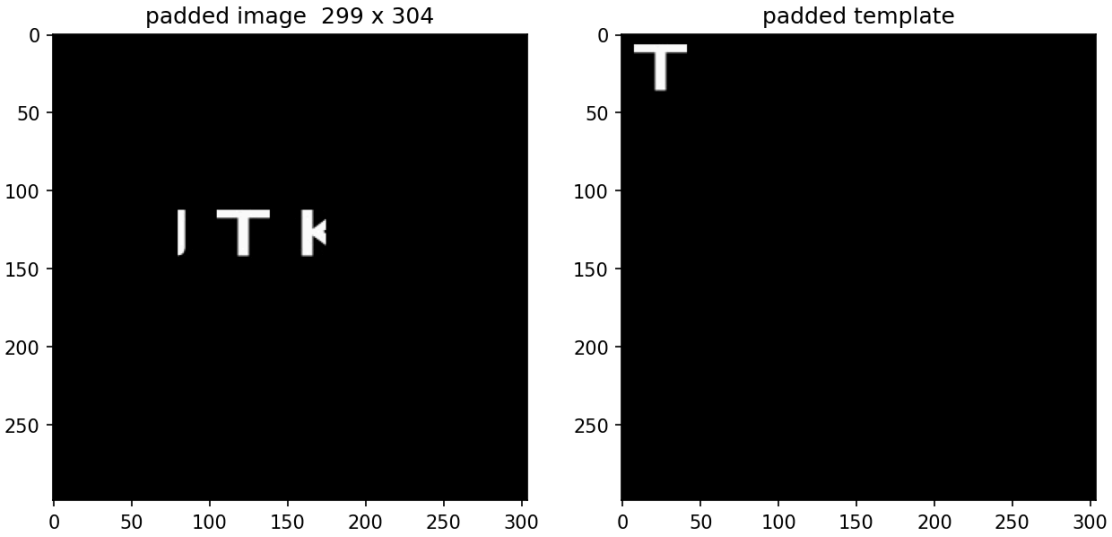


图 15　fig02_0405_padded.png：例 4.11 布局下零填充后的图像（左）与模板（右），对应图 4.41(c) 与 (d)。

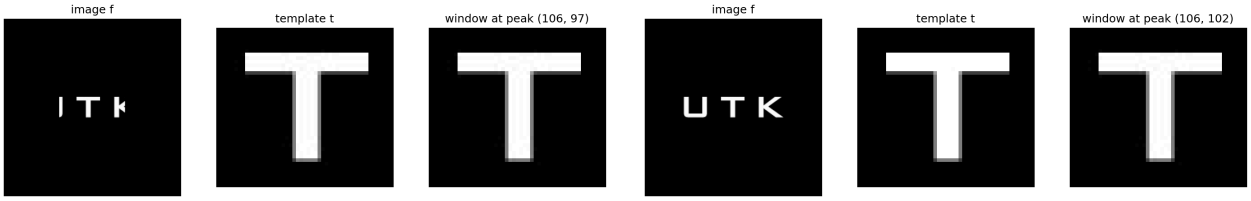


图 16　fig_ex_a_detect.png（左，重排素材）与 fig_sup_a_detect.png（右，附件素材）：图像 f、模板 t，以及相关峰值处切出的窗口。两者都与模板逐像素一致。

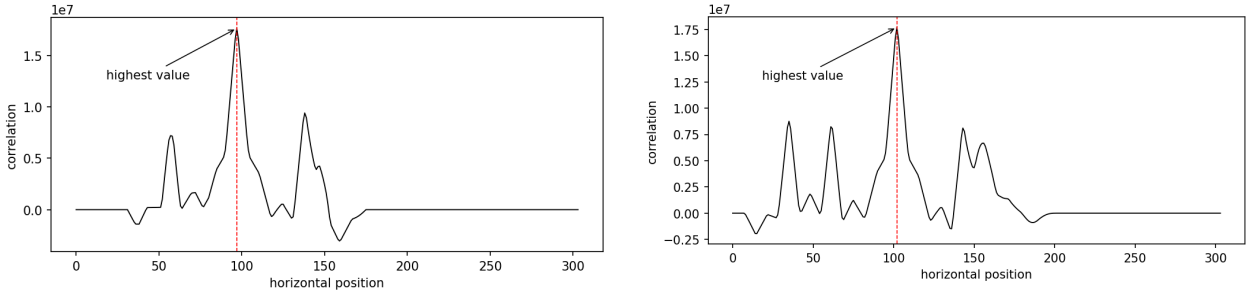


图 17　fig_ex_f_profile.png（左）与 fig_sup_f_profile.png（右）：通过最高值的水平灰度剖面线，对应图 4.41(f)。

结果。 两组素材的相关函数都出现一个尖锐的单峰：重排素材的峰/均值比高达 110，附件素材为 63.7，而其它位置的取值都低一个数量级以上（17 的剖面线）。峰值处切出的窗口与模板逐像素相同，归一化互相关系数（NCC）为 1.0000。这与教材“相关函数的最大值正好出现在模板所在的点上”的结论一致。

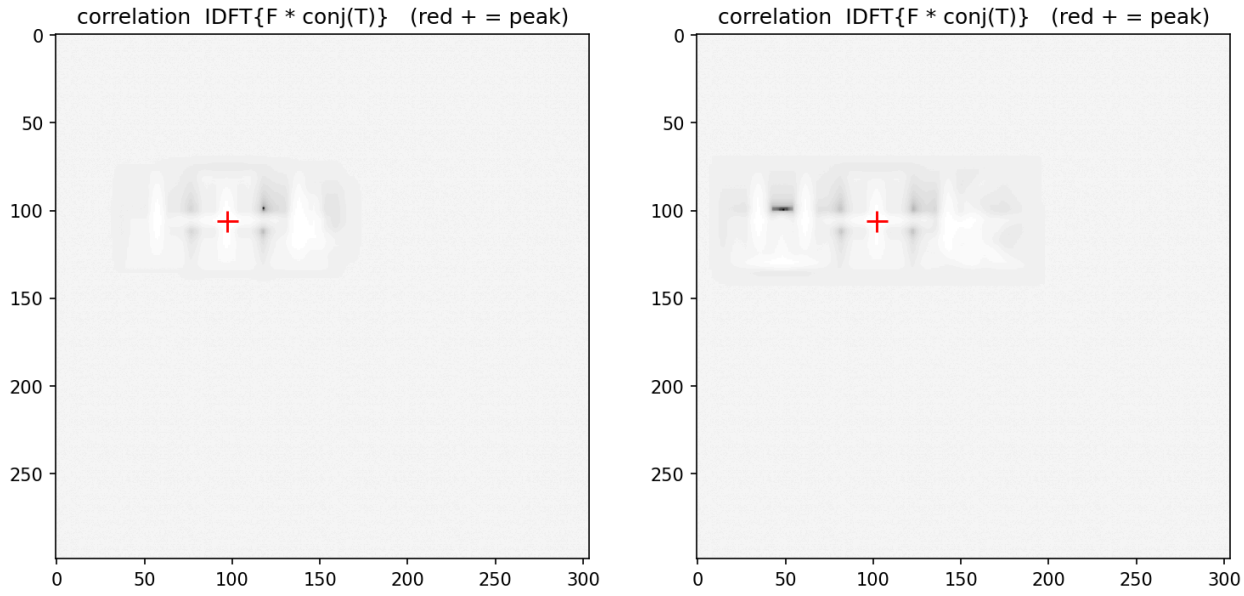


图 18　fig_ex_e_corr.png（左）与 fig_sup_e_corr.png（右）：相关函数 g 以图像方式显示（灰度取对数拉伸），红色十字为峰值，对应图 4.41(e)。

关于显示。相关函数大部分区域接近 0，峰值却高出几个数量级，直接按线性灰度显示会几乎全黑。我们把灰度做对数拉伸后再显示（18），既保留峰值又能看见周围的次级结构；剖面线（17）则用原始数值，可以看清主峰与旁瓣的数量级差异。

2.4 验证（`test/testing_correlation.py`）

在做定位之前，先验证相关本身的正确性。所有图像以 `float64` 载入。

检查项	最大误差	结论
DFT 往返 (8×8)	1.3×10^{-15}	PASS
DFT 往返 (6×10)	1.8×10^{-15}	PASS
随机图的峰位于真实位置	0	PASS
峰值窗口等于模板	0	PASS
DFT 相关 = 直接相关	3×10^{-14}（相对峰值）	PASS
端到端 NCC = 1	2.0×10^{-15}	PASS
退化尺寸 (1×1)、(1×5)、(5×1)、(3×3)	$\leq 1.7 \times 10^{-16}$	PASS

“DFT 相关 = 直接相关”是把 DFT 结果与按定义逐窗口点积的结果比较；两者之差相对峰值只有 3×10^{-14}，属于浮点舍入水平，说明实现与相关的定义一致。

2.5 对其他图像的推广

在不改动算法的情况下，把同一套代码用于另外两幅测试图像（`test/test_1.bmp`，即教材字符测试图；`test/test_2.bmp`，戴图案纸生日帽的幼童灰度图）。模板直接从图像中切出，因此真实位置已知。

图像	尺寸	模板	真实位置	找到的位置
`test_1.bmp`	688×688	140×160	$(300, 250)$	$(300, 250)$
`test_2.bmp`	812×812	180×200	$(60, 300)$	$(60, 300)$

两组都精确找回了模板（NCC = 1.0000）。

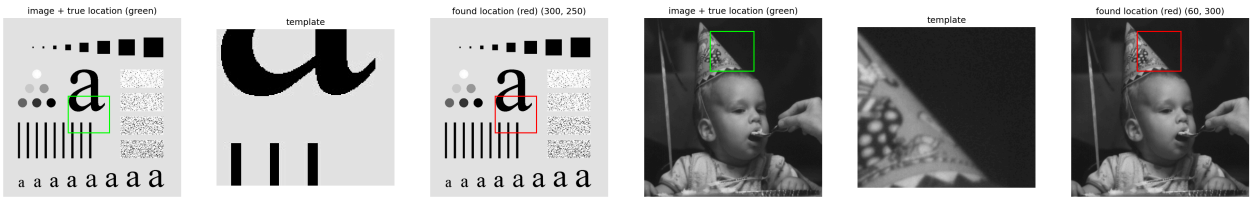


图 19　test/test_1_results/ 与 test/test_2_results/ 中的 detection.png：绿框为切出模板的真实位置，红框为相关找到的位置，两者重合。

3 扩展实验与性能

必做内容比较简单，因此我们还研究了该方法的其他性质。所有实验都在 `src/extra.py` 中，每项计时取 3 次运行中的最好值。

3.1 直接相关与 DFT 相关

按定义直接做相关是两层循环的 $O(MNCD)$ 算法。对 256×256 的图与 44×49 的模板，两者给出相同的结果（最大绝对差 6×10^{-7}，相对峰值约 3×10^{-14}），而耗时分别为：

方法	时间（s）	加速比
DFT 相关（两次变换 + 一次逐点乘）	0.026	`1.0x`
直接相关（按定义）	0.318	`12.2x`

DFT 把相关从 $O(MNCD)$ 降到两次二维变换的成本，模板越大优势越明显。

3.2 时间随图像尺寸的增长

在 $n \times n$ 随机图上用固定的 32×32 模板测时间：

n	64	128	256	512	1024
时间（s）	0.002	0.006	0.024	0.088	0.405

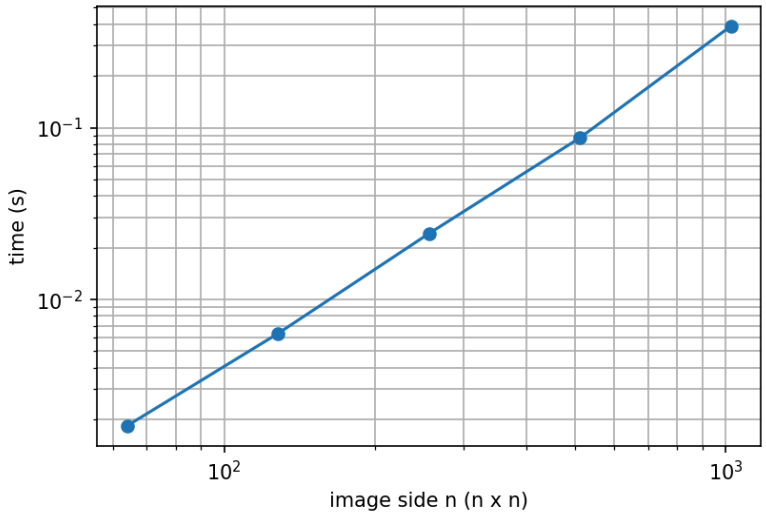


图 20　ext2_speed.png：DFT 相关的耗时随图像边长的变化（双对数坐标）。

双对数斜率为 1.9：与“两次变换”的预期一致。

3.3 图像均值不为零时的朴素相关

相关函数里的直流项 $\overline{F(0,0)}\cdot T(0,0)$ 与图像均值成正比。当图像整体偏亮（均值较大）时，这一项会压过真正匹配的贡献，使相关最大值跑到“亮块与亮块重叠最多”的位置，而不是模板所在处。把图像反相（$255-f$，变成亮背景）后：

相关形式	峰值位置	峰值 / 均值
朴素相关（教材写法）	$(293, 302)$，即右下角	1.4
均值去除后（归一化）	$(106, 97)$，正确	很大

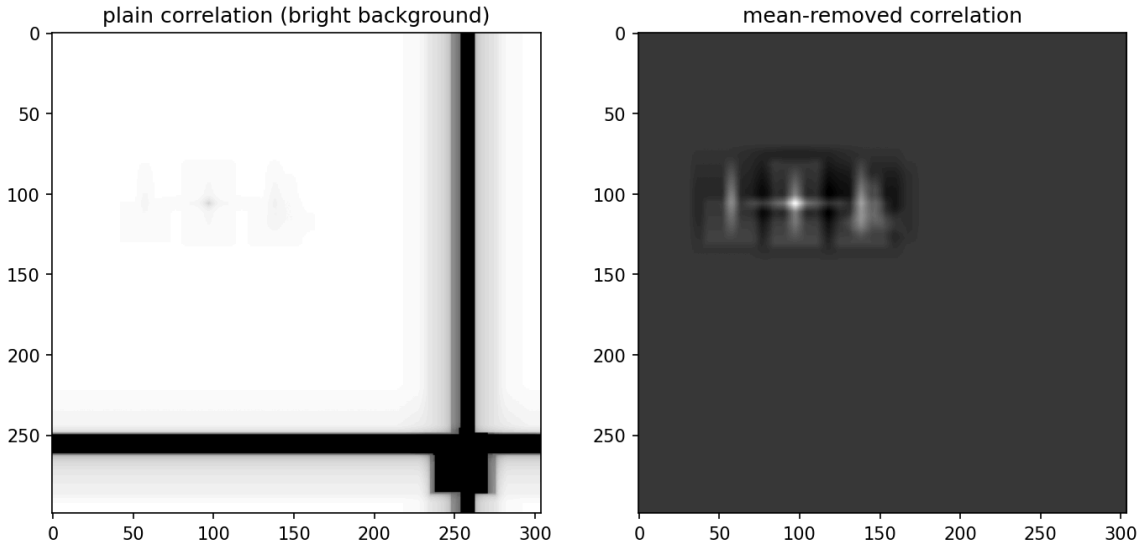


图 21　ext3_normalized.png：亮背景下朴素相关（左，峰值在角落）与均值去除后相关（右，峰值在物体处）的对比。

这种做法就是把模板（以及窗口）减去自身均值再相关，即归一化互相关。教材的例子中图像以黑底为主、均值为 5.2，直流项很小，所以朴素相关同样能得到尖锐的单峰；但对一般图像，均值去除更稳健。

3.4 同一个模板、另一个目标

相关只依赖“模板与窗口是否相似”，与模板的内容无关。把模板换成同一个 “UTK” 块里的 “K” 后，同一段代码把它定位到 $(114, 148)$：

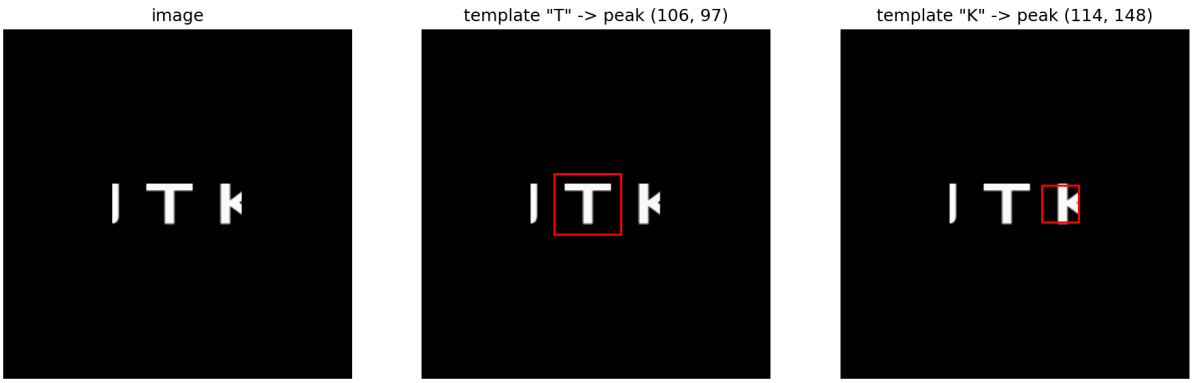


图 22　ext4_two_templates.png：同一套相关代码分别定位 “T”（中）与 “K”（右）。

3.5 非方形、含奇数边长的图像

在 190×230 与 189×229（含奇数边长）的裁剪图上重复定位，NCC 均为 1.0000。相关性由 DFT 矩阵直接按图像的高、宽构造，既不需要方形，也不需要边长是 2 的幂，更不需要把尺寸补到某个“方便”的值；只需要零填充满足 $P\geq A+C-1$ 即可。

3.6 习题 4.22 与 4.23

习题 **4.23**（取另一个变换的共轭）。 用 $\overline{F}\cdot T$ 代替 $F\cdot\overline{T}$，峰值从 $(106,97)$ 移到 $(193,207)$，满足

$$(106+193-1,\,97+207-1)=(298,303)=(P-1,Q-1),$$

即峰值关于相关数组中心与原来的峰值镜像对称，峰值大小完全相同（2.129×10^{7}）。这是意料之中的：取共轭把相关翻转成关于中心的反射，所以位置镜像、大小不变。这也说明 2.3 中两种写法只是整体共轭的差别。

习题 **4.22**（省去 $(-1)^u$ 因子）。 如果把定义里 $t(s-x,t-y)$ 的符号处理错，等价于在频域少乘一个 $(-1)^u$。此时峰值跑到 $(282,97)$（数组顶部），峰值降到 1.415×10^{6}，仅为正确值的 6.6%——相关已经不能用来定位模板了。

4 关键代码

4.1 `src/correlation.py`

```python
def pad_to(f, shape):
    """Zero-pad (bottom/right) f to the given (P, Q) shape."""
    P, Q = shape
    M, N = f.shape
    g = np.zeros((P, Q), dtype=np.float64)
    g[:M, :N] = f
    return g

def circular_correlate(f, tpl, pad, subtract_mean=False):
    """g = IDFT{ F * conj(T) } on the zero-padded arrays."""
    fp = pad_to(f, pad)
    tp = pad_to(tpl, pad)
    if subtract_mean:                       # remove the DC term
        tp[: tpl.shape[0], : tpl.shape[1]] -= tpl.mean()
        fp = fp - fp.mean()
    return np.real(idft2d(dft2d(fp) * np.conj(dft2d(tp))))

def linear_correlate(f, tpl, pad, subtract_mean=False):
    """Exact linear correlation; needs P >= M + C - 1, Q >= N + D - 1."""
    M, N = f.shape
    C, D = tpl.shape
    if pad[0] < M + C - 1 or pad[1] < N + D - 1:
        raise ValueError("pad too small")
    return circular_correlate(f, tpl, pad, subtract_mean)

def find_target(corr):
    """Location (row, col) of the maximum: the template's top-left corner."""
    idx = int(np.argmax(corr))
    return idx // corr.shape[1], idx % corr.shape[1]
```

4.2 `src/main.py`（必做部分）

```python
img = load("example_a.png")               # 256 x 256
tpl = load("example_b.png")               # 44 x 49 template
(M, N), (C, D) = img.shape, tpl.shape
pad = (M + C - 1, N + D - 1)              # 299 x 304, the exact minimum

corr = linear_correlate(img, tpl, pad, subtract_mean=True)
loc  = find_target(corr)                  # -> (106, 97)
win  = match_at(img, tpl, loc, subtract_mean=True)
```

4.3 src/extra.py（扩展与习题）

```
# Exercise 4.23: exchange the two transforms -> mirrored peak
F = dft2d(pad_to(img, pad)); T = dft2d(pad_to(tpl, pad))
full    = np.real(idft2d(F * np.conj(T)))
swapped = np.real(idft2d(np.conj(F) * T))

# Exercise 4.22: the factor (-1)^u omitted
u = np.arange(pad[0])[:, None]
omitted = np.real(idft2d(F * np.conj(T) * (-1.0) ** u))

# Bright background: the plain correlation is defeated by the DC term
plain = linear_correlate(bright, tpl, pad, subtract_mean=False)
norm  = linear_correlate(bright, tpl, pad, subtract_mean=True)
```

5 实验结果

我们用自实现的 DFT 完成了二维相关，并把模板在图像中定位。按教材式子的形式 $g = \mathrm{IDFT}\big(F \cdot \overline{T}\big)$，在例 4.11 的素材上相关函数出现尖锐的单峰（峰值 / 均值 $= 110$），峰值处切出的窗口与模板逐像素相同（NCC $= 1.0000$），复现了图 4.41(c)–(f)。同一套代码在附件提供的素材、教材字符测试图与一幅灰度照片上都精确找回模板，也适用于非方形、含奇数边长的图像。数值上，DFT 相关与按定义逐窗口点积的结果之差相对峰值仅 3×10^{-14}，而速度快约 12 倍。此外我们验证了习题 4.23（取另一个变换的共轭会把峰值镜像、大小不变）与习题 4.22（丢掉 $(-1)^u$ 因子后峰值降到 6.6%、无法定位），并指出当图像均值较大时朴素相关会被直流项压垮、需要先去除均值。